



Computer Networks

Ch.2: Application Layer

Prepared By
Dr. Ibrahim Attiya

© 2024 NMU

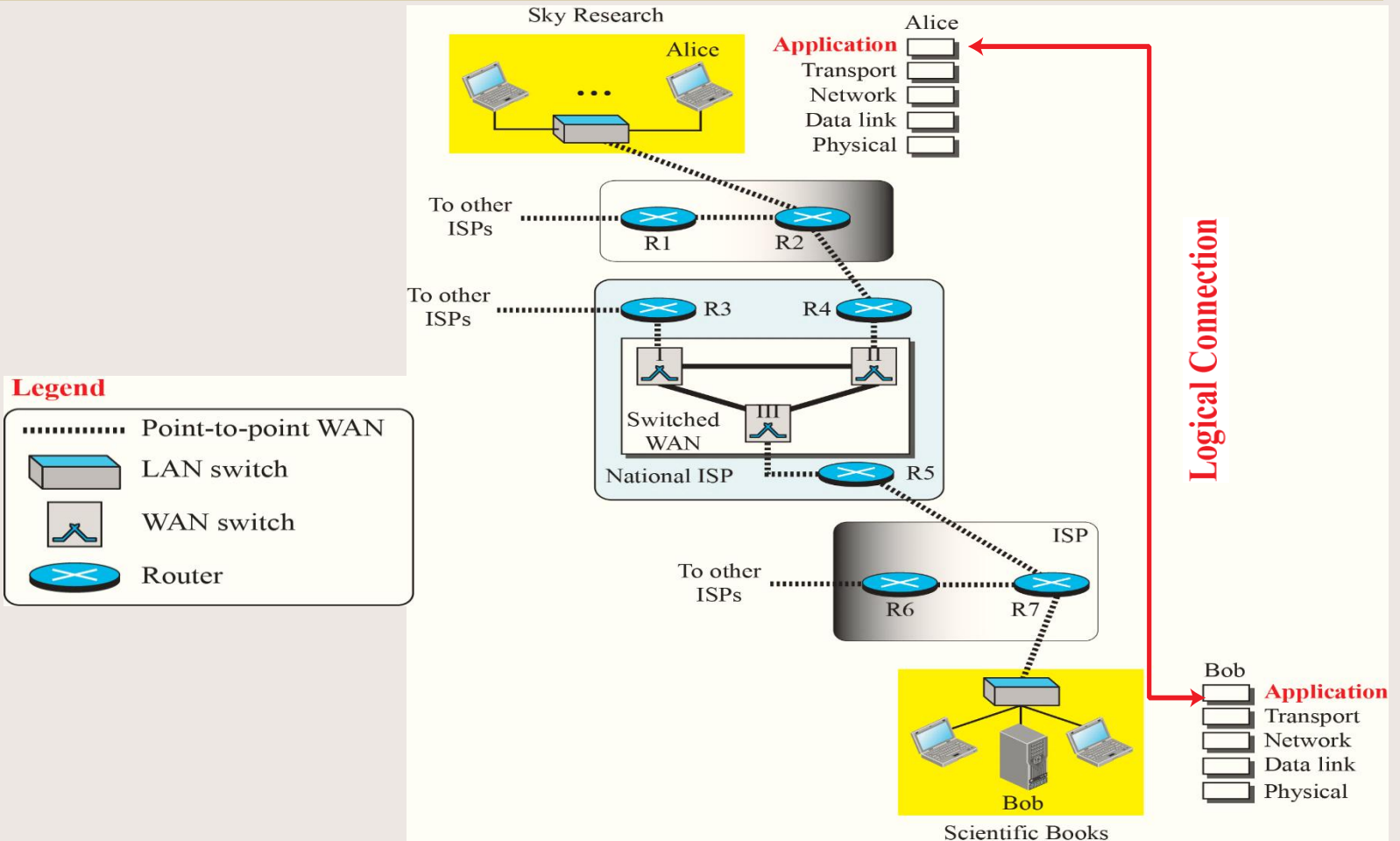
Ch.2: Outline

- ☐ Introduction
- ☐ Application-Layer Paradigms
- ☐ Client-Server Paradigm
- ☐ Socket Interface
- ☐ Standard Applications
- ☐ Peer-to-Peer Paradigm
- ☐ Socket Interface Programming
- ☐ Standard Client-Server Applications

Introduction

- The whole Internet, was designed and developed to provide services at the application layer of the TCP/IP protocol suite.
- The other four layers are there to make these services possible.
- Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages.

Introduction



Logical connection at the application layer

Standard and Nonstandard Protocols

- To provide a smooth operation of the Internet, the protocols used in the first four layers of the TCP/IP suite need to be *standardized* and *documented*.
- They normally become part of the package that is included in operating systems such as Windows or UNIX.
- To be flexible, however, the application layer protocols can be both standard and nonstandard.

Nonstandard Application-Layer Protocols

- A programmer can create a nonstandard application layer program if he/she can write two programs that provide service to the user by interacting with the transport layer.
- The creation of a nonstandard protocol that does not even need the approval of the Internet authorities if privately used, has made the Internet so popular in the world.
 - A private company can create a new customized application protocol to communicate with all of its offices around the world.

Application-Layer Paradigms

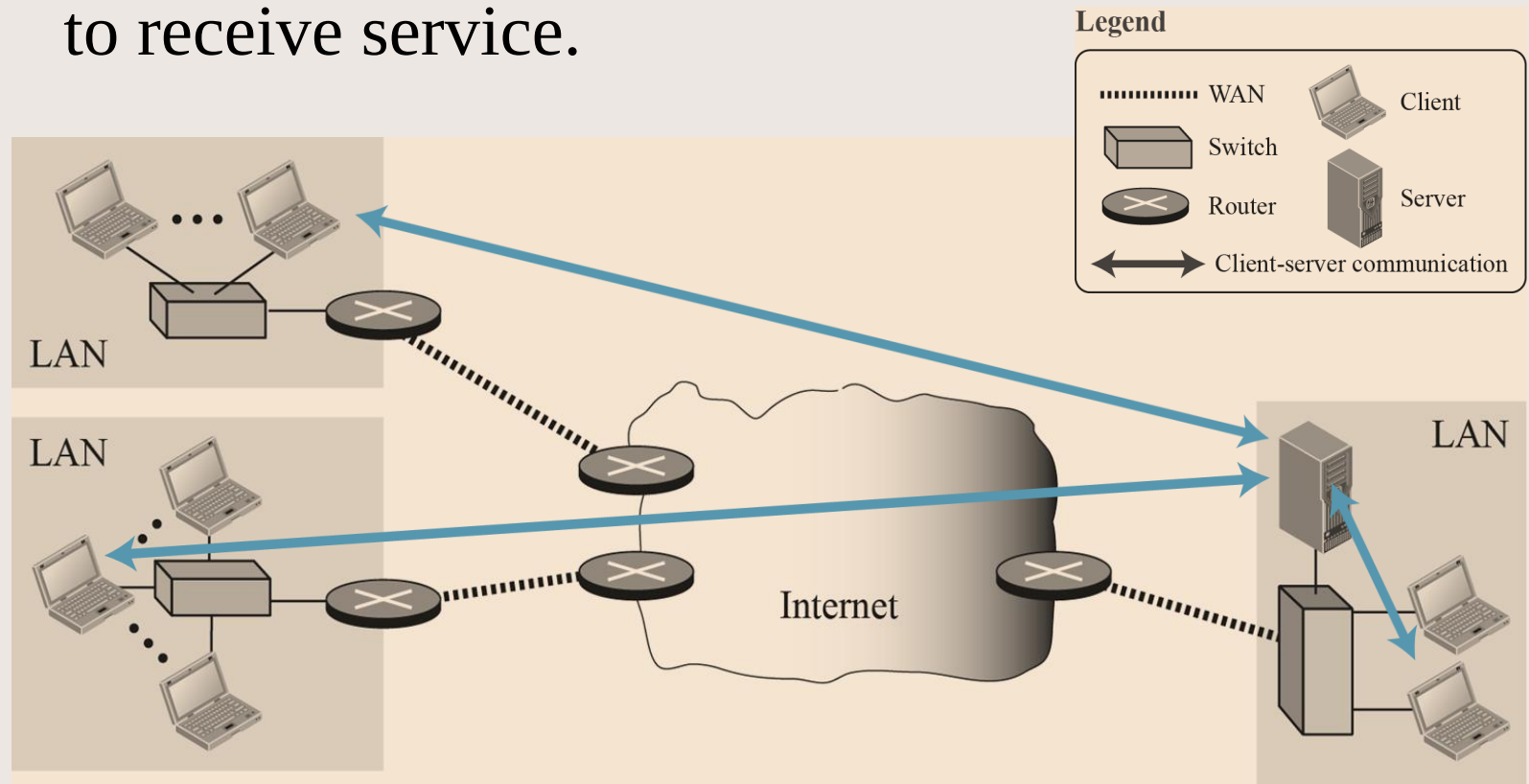
- Two programs need to send messages to each other through the Internet infrastructure.
- Should both application programs be able to request services and provide services, or should the application programs just do one or the other?
- Two paradigms have been developed during the lifetime of the Internet to answer this question: the *client-server paradigm* and the *peer-to-peer paradigm*.

Traditional Paradigm: Client-Server

- The traditional paradigm is called the *client-server paradigm*.
- It was the most popular paradigm until a few years ago.
- In this paradigm, the service provider is an application program, called the *server* process; it runs continuously, waiting for another application program, called the *client* process, to make a connection through the Internet and ask for service.

Traditional Paradigm: Client-Server

- The server process must be running all the time; the client process is started when the client needs to receive service.



Traditional Paradigm: Client-Server

- One problem with this paradigm is that the concentration of the communication load is on the shoulder of the server, which means the server should be a powerful computer.
- Another problem is that the service must always return some type of income for the server in order to encourage such an arrangement.
- Several traditional services are still using this paradigm, including the WWW and its vehicle HTTP, FTP, SSH, e-mail, and so on.

New Paradigm: Peer-to-Peer (P2P)

- In this paradigm, there is no need for a server process to be running all the time and waiting for the client processes to connect. The responsibility is shared between peers.
- A computer connected to the Internet can provide service at one time and receive service at another time.
- A computer can even provide and receive services at the same time.
- One of the areas that really fits in this paradigm is the Internet telephony.

New Paradigm: Peer-to-Peer (P2P)

- The main challenge has been security; it is more difficult to create secure communication between distributed services than between those controlled by some dedicated servers.
- The other challenge is applicability; it appears that not all applications can use this new paradigm.
- There are some new applications, such as BitTorrent, Skype, IPTV, and Internet telephony, that use this paradigm.

Client-Server Paradigm

- In this paradigm, communication at the application layer is between two running application programs called **processes**: a *client* and a *server*.
- A client is a running program that initializes the communication by sending a request; a server is another application program that waits for a request from a client.
- The server handles the request received from a client, prepares a result, and sends the result back to the client.

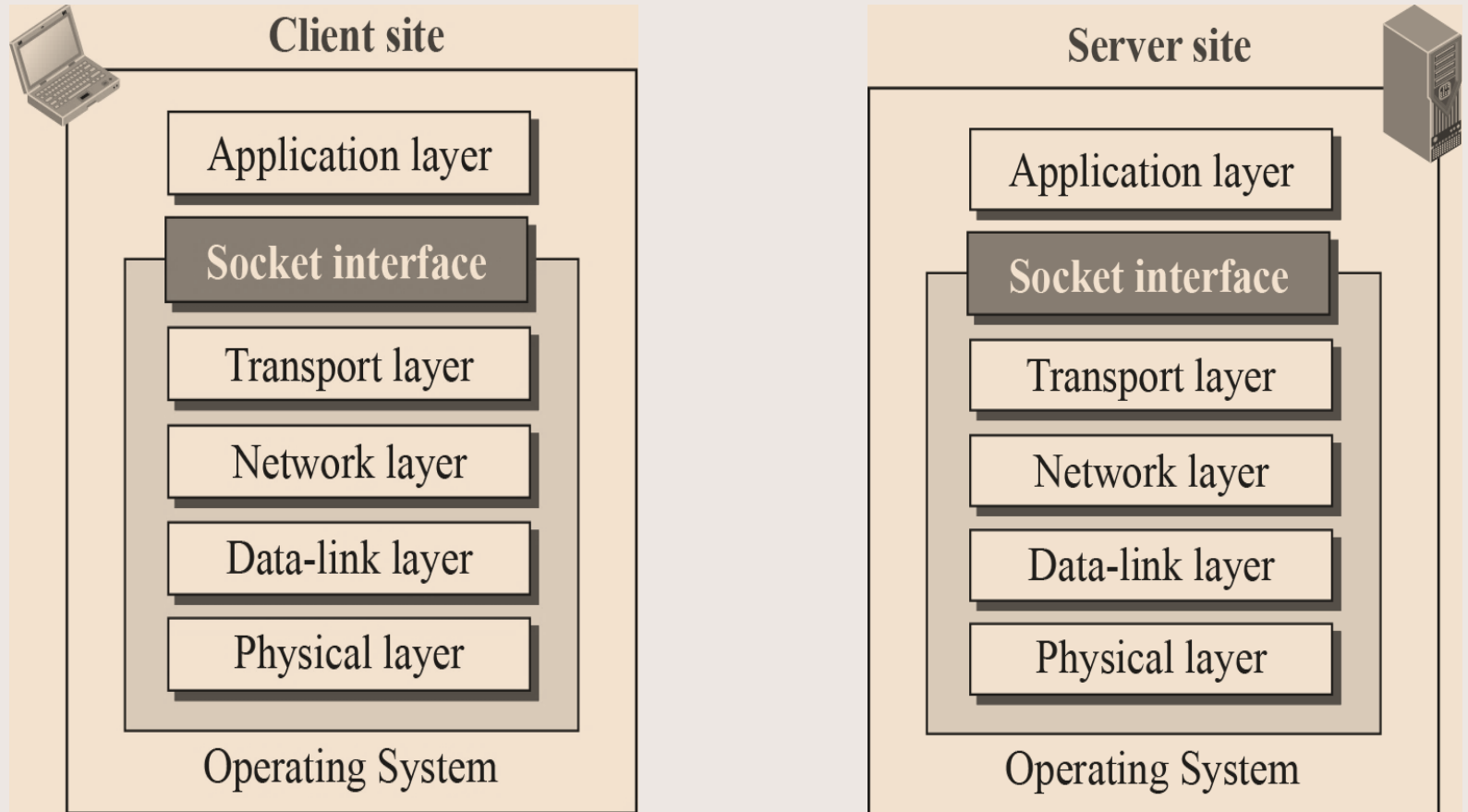
Application Programming Interface

- If we need a process to be able to communicate with another process, we need a new set of instructions to tell the lowest four layers of the TCP/IP suite to open the connection, send and receive data from the other end, and close the connection.
- A set of instructions of this kind is normally referred to as **Application Programming Interface (API)**.
- Several APIs have been designed for communication. Among them is the **socket interface**.

Socket Interface

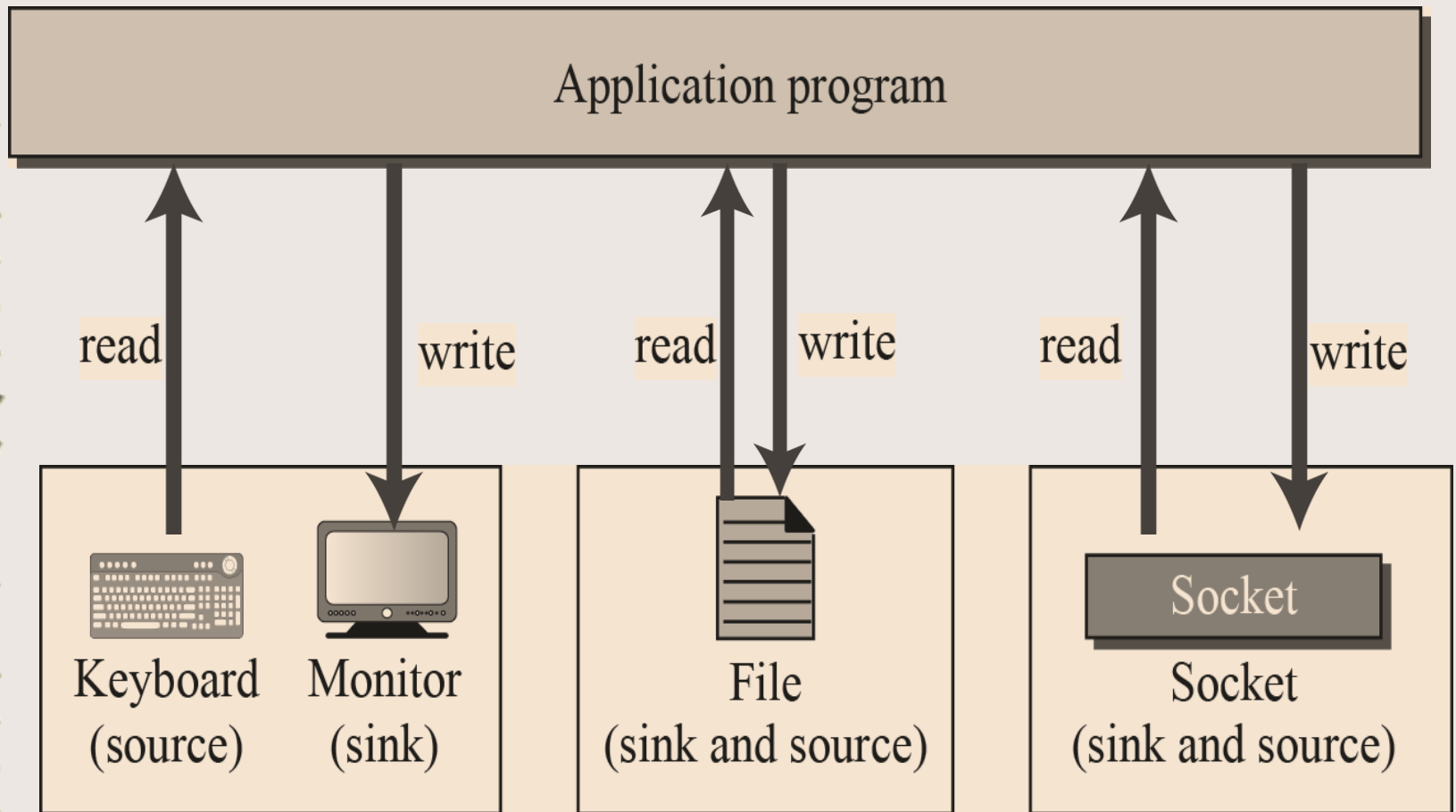
- The socket interface is a set of instructions that provide communication between the application layer and the operating system.
- It is a set of instructions that can be used by a process to communicate with another process.
- Sockets are used in network programming and are essential for applications that require internet or network connectivity, such as web browsers, email clients, or file-sharing applications.

Socket Interface



Position of the socket interface

Socket Interface



A Sockets used like other sources and sinks

Socket Interface

- The interaction between a client and a server is two-way communication.
- In a two-way communication, we need a pair of addresses: local (sender) and remote (receiver).
- Since communication in the client-server paradigm is between two sockets, we need a pair of socket addresses for communication: a *local socket address* and a *remote socket address*.

Socket Addresses

- A socket address should first define the computer on which a client or a server is running.
- A computer on the Internet is uniquely defined by its **IP address**, a 32-bit integer in the current Internet version.
- However, several client or server processes may be running at the same time on a computer, which means that we need another identifier to define the specific client or server involved in the communication.

Socket Addresses

- Hence, an application program can be defined by a **port number**, a 16-bit integer.
 - Ex., port numbers: —HTTP server: 80 — Mail server: 25
- This means that a socket address should be a combination of an ***IP address*** and a ***port number***.



Common Socket Interface Functions

- **socket():** Creates a new socket.
- **bind():** Assigns an IP address and port to the socket.
- **listen():** Prepares a socket to accept incoming connections (server-side).
- **connect():** Establishes a connection to a server (client-side).
- **send()** and **receive():** Send or receive data through the socket.
- **close():** Closes the socket and ends the connection.

Uses of Sockets

- **Web Servers and Browsers:** Sockets are used to send and receive HTTP requests and responses.
- **Messaging Services:** Enable chat and real-time messaging by managing continuous connections.
- **File Transfer:** Used in protocols like FTP to establish data exchange between devices.

A spiral-bound notebook with a brown cover and a white page. The spiral binding is on the left side. The text "Any Questions?" is written in a black, cursive font in the center of the page.

Any Questions?